

assignment-numpy-assignment

March 20, 2025

```
[1]: import numpy as np
arr = np.array([0,1,2,3,4,5])
print(arr.dtype)

arr = np.arange(6)
print(arr.dtype)
```

```
int32
int32
```

```
[2]: import numpy as np

arr = np.array([1.5, 2.6, 3.7])

# Check if the dtype is float64
if arr.dtype == np.float64:
    print("The array data type is float64.")
else:
    print("The array data type is not float64.")
```

The array data type is float64.

```
[3]: import numpy as np

arr = np.array([1+2j, 3+4j, 5+6j], dtype=np.complex128)

print(arr)
print(arr.dtype) # Should print complex128
```

```
[1.+2.j 3.+4.j 5.+6.j]
complex128
```

```
[4]: import numpy as np

# Creating an integer array
arr = np.array([1, 2, 3, 4, 5])
```

```

# Converting to float32
arr_float = arr.astype(np.float32)

# Print the new array and its data type
print(arr_float)
print(arr_float.dtype) # Should print float32

```

```

[1. 2. 3. 4. 5.]
float32

```

```

[5]: import numpy as np

# Creating a float64 array
arr = np.array([1.23456789, 2.98765432, 3.14159265], dtype=np.float64)

# Converting to float32
arr_float32 = arr.astype(np.float32)

# Print the new array and its data type
print(arr_float32)
print(arr_float32.dtype) # Should print float32

```

```

[1.2345679 2.9876542 3.1415927]
float32

```

```

[6]: import numpy as np

def array_attributes(arr):
    """
    Returns the shape, size, and data type of a given NumPy array.

    Parameters:
        arr (numpy.ndarray): Input NumPy array

    Returns:
        tuple: (shape, size, dtype) of the array
    """
    return arr.shape, arr.size, arr.dtype

# Example usage
arr = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.int32)
print(array_attributes(arr)) # Output: ((2, 3), 6, dtype('int32'))

```

```

((2, 3), 6, dtype('int32'))

```

```

[7]: import numpy as np

```

```
def array_dimension(arr):
    """
    Returns the number of dimensions (rank) of a given NumPy array.

    Parameters:
        arr (numpy.ndarray): Input NumPy array

    Returns:
        int: The number of dimensions of the array
    """
    return arr.ndim

# Example usage
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(array_dimension(arr)) # Output: 2
```

2

```
[8]: import numpy as np

def item_size_info(arr):
    """
    Returns the item size (in bytes) and the total size (in bytes) of a given
    NumPy array.

    Parameters:
        arr (numpy.ndarray): Input NumPy array

    Returns:
        tuple: (item size in bytes, total size in bytes)
    """
    return arr.itemsize, arr.nbytes

arr = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.int32)
print(item_size_info(arr)) # Output: (4, 24) for int32 array
```

(4, 24)

```
[9]: import numpy as np

def array_strides(arr):
    """
    Returns the strides of a given NumPy array.

    Parameters:
        arr (numpy.ndarray): Input NumPy array
```

```

Returns:
    tuple: Strides of the array
    """
    return arr.strides

# Example usage
arr = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.int32)
print(array_strides(arr)) # Output: (12, 4) for int32 array

```

(12, 4)

```

[10]: import numpy as np

def shape_stride_relationship(arr):
    """
    Returns the shape and strides of a given NumPy array.

    Parameters:
        arr (numpy.ndarray): Input NumPy array

    Returns:
        tuple: (shape, strides) of the array
    """
    return arr.shape, arr.strides

# Example usage
arr = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.int32)
print(shape_stride_relationship(arr)) # Output: ((2, 3), (12, 4)) for int32
    ↪ array

```

((2, 3), (12, 4))

```

[14]: import numpy as np
def create_zeros_array(n):

    return np.zeros(n)

arr = create_zeros_array(5)
print(arr)
print(arr.dtype)

```

```

Cell In[14], line 4
    return np.zeros(n)
    ^

```

IndentationError: expected an indented block after function definition on line 4

```
[15]: import numpy as np

def create_ones_matrix(rows, cols):
    """
    Creates a 2D NumPy array filled with ones of size rows x cols.

    Parameters:
        rows (int): Number of rows
        cols (int): Number of columns

    Returns:
        numpy.ndarray: A 2D array filled with ones
    """
    return np.ones((rows, cols))

# Example usage
matrix = create_ones_matrix(3, 4)
print(matrix)
print(matrix.dtype) # Default dtype is float64
```

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
float64
```

```
[16]: import numpy as np

def generate_range_array(start, stop, step):
    """
    Creates a NumPy array with values ranging from start to stop (exclusive)
    with a given step.

    Parameters:
        start (int): The starting value of the sequence.
        stop (int): The end value (exclusive).
        step (int): The step size between values.

    Returns:
        numpy.ndarray: A NumPy array containing the generated range.
    """
    return np.arange(start, stop, step)

# Example usage
arr = generate_range_array(1, 10, 2)
print(arr) # Output: [1 3 5 7 9]
```

```
[1 3 5 7 9]
```

```
[17]: import numpy as np

def generate_linear_space(start, stop, num):
    """
    Generates a NumPy array with num equally spaced values between start and
    stop (inclusive).

    Parameters:
        start (float): The starting value of the sequence.
        stop (float): The ending value of the sequence.
        num (int): Number of equally spaced values to generate.

    Returns:
        numpy.ndarray: A NumPy array with linearly spaced values.
    """
    return np.linspace(start, stop, num)

# Example usage
arr = generate_linear_space(0, 10, 5)
print(arr) # Output: [ 0.  2.5  5.  7.5 10. ]
```

```
[ 0.  2.5  5.  7.5 10. ]
```

```
[18]: import numpy as np

def create_identity_matrix(n):
    """
    Creates an identity matrix of size n x n.

    Parameters:
        n (int): The size of the identity matrix.

    Returns:
        numpy.ndarray: An n x n identity matrix.
    """
    return np.eye(n)

# Example usage
matrix = create_identity_matrix(4)
print(matrix)
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

```
[19]: import numpy as np

def list_to_numpy_array(py_list):
    """
    Converts a Python list into a NumPy array.

    Parameters:
        py_list (list): The input Python list.

    Returns:
        numpy.ndarray: A NumPy array created from the list.
    """
    return np.array(py_list)

# Example usage
my_list = [1, 2, 3, 4, 5]
arr = list_to_numpy_array(my_list)
print(arr)          # Output: [1 2 3 4 5]
print(type(arr))    # Output: <class 'numpy.ndarray'>
print(arr.dtype)    # Output: int32 or int64 (depending on the system)
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
int32
```

```
[20]: import numpy as np

# Create an original NumPy array
arr = np.array([10, 20, 30, 40, 50], dtype=np.int32)

# Create a new view of the array
arr_view = arr.view()

# Modify the view and observe the effect on the original array
arr_view[0] = 99

# Print both arrays
print("Original Array:", arr)          # Output: [99 20 30 40 50] (modified)
print("View Array:", arr_view)        # Output: [99 20 30 40 50]

# Check if they share the same memory
print("Same Memory?", arr.base is arr_view.base) # Output: True
```

```
Original Array: [99 20 30 40 50]
View Array: [99 20 30 40 50]
Same Memory? False
```

```
[21]: import numpy as np

def concatenate_arrays(arr1, arr2, axis=0):
    """
    Concatenates two NumPy arrays along a specified axis.

    Parameters:
        arr1 (numpy.ndarray): First input array.
        arr2 (numpy.ndarray): Second input array.
        axis (int): The axis along which to concatenate (default is 0).

    Returns:
        numpy.ndarray: The concatenated array.
    """
    return np.concatenate((arr1, arr2), axis=axis)

# Example usage
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])

# Concatenating along axis 0 (rows)
result1 = concatenate_arrays(arr1, arr2, axis=0)
print("Concatenated along axis 0:\n", result1)

# Concatenating along axis 1 (columns)
result2 = concatenate_arrays(arr1, arr2, axis=1)
print("Concatenated along axis 1:\n", result2)
```

Concatenated along axis 0:

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

Concatenated along axis 1:

```
[[1 2 5 6]
 [3 4 7 8]]
```

```
[22]: import numpy as np

# Create two NumPy arrays with different shapes
arr1 = np.array([[1, 2], [3, 4]])    # Shape (2, 2)
arr2 = np.array([[5], [6]])          # Shape (2, 1)

# Concatenating horizontally (axis=1)
result = np.concatenate((arr1, arr2), axis=1)

# Print the result
```

```
print("Concatenated Array:\n", result)
```

Concatenated Array:

```
[[1 2 5]
 [3 4 6]]
```

```
[23]: import numpy as np

def vertical_stack_arrays(arr_list):
    """
    Vertically stacks multiple NumPy arrays.

    Parameters:
        arr_list (list of numpy.ndarray): List of NumPy arrays to be stacked.

    Returns:
        numpy.ndarray: A single vertically stacked array.
    """
    return np.vstack(arr_list)

# Example usage
arr1 = np.array([[1, 2], [3, 4]]) # Shape (2, 2)
arr2 = np.array([[5, 6]])         # Shape (1, 2)
arr3 = np.array([[7, 8]])         # Shape (1, 2)

# Stacking multiple arrays vertically
result = vertical_stack_arrays([arr1, arr2, arr3])

# Print the result
print("Vertically Stacked Array:\n", result)
```

Vertically Stacked Array:

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

```
[24]: import numpy as np

def create_integer_array(start, end, step):
    """
    Create an array of integers within a specified range (inclusive) with a
    given step size.

    Parameters:
        start (int): The starting integer.
        end (int): The ending integer (inclusive).
    """
```

```

    step (int): The step size.

    Returns:
    np.ndarray: A NumPy array containing the integers.
    """
    return np.arange(start, end + step, step) if step > 0 else np.arange(start,
↪end - step, step)

# Example usage:
array = create_integer_array(1, 10, 2)
print(array) # Output: [ 1  3  5  7  9 ]

```

```
[ 1  3  5  7  9 11]
```

```

[25]: import numpy as np

def generate_equal_spaced_array():
    """
    Generate an array of 10 equally spaced values between 0 and 1 (inclusive).

    Returns:
    np.ndarray: A NumPy array containing the values.
    """
    return np.linspace(0, 1, 10)

# Example usage:
array = generate_equal_spaced_array()
print(array)

```

```
[0.          0.11111111 0.22222222 0.33333333 0.44444444 0.55555556
 0.66666667 0.77777778 0.88888889 1.          ]
```

```

[26]: import numpy as np

def generate_log_spaced_array():
    """
    Generate an array of 5 logarithmically spaced values between 1 and 1000,
↪(inclusive).

    Returns:
    np.ndarray: A NumPy array containing the values.
    """
    return np.logspace(0, 3, 5) # 10^0 to 10^3, 5 values

# Example usage:
array = generate_log_spaced_array()
print(array)

```

```
[ 1.          5.62341325  31.6227766   177.827941   1000.          ]
```

```
[28]: import numpy as np
import pandas as pd

def create_random_dataframe():
    """
    Create a Pandas DataFrame with 5 rows and 3 columns containing random
    integers between 1 and 100.

    Returns:
    pd.DataFrame: A Pandas DataFrame with random values.
    """
    data = np.random.randint(1, 101, size=(5, 3)) # Generate a 5x3 array of
    random integers
    df = pd.DataFrame(data, columns=['A', 'B', 'C']) # Create a DataFrame with
    column labels
    return df

# Example usage:
df = create_random_dataframe()
print(df)
```

```
   A  B  C
0  58  20  74
1  41  99  13
2  58  63  94
3  51  79  68
4  79  21  89
```

```
[29]: import numpy as np
import pandas as pd

def replace_negatives_with_zero(df, column_name):
    """
    Replace all negative values in a specific column of a Pandas DataFrame with
    zeros.

    Parameters:
    df (pd.DataFrame): The input DataFrame.
    column_name (str): The name of the column to modify.

    Returns:
    pd.DataFrame: The modified DataFrame with negative values replaced by zeros.
    """
    df[column_name] = np.where(df[column_name] < 0, 0, df[column_name]) #
    Replace negatives with 0
```

```

    return df

# Example usage:
data = {'A': [10, -5, 30, -2, 50], 'B': [7, -3, 8, 15, -1]}
df = pd.DataFrame(data)

print("Before:")
print(df)

df = replace_negatives_with_zero(df, 'A')

print("\nAfter:")
print(df)

```

Before:

	A	B
0	10	7
1	-5	-3
2	30	8
3	-2	15
4	50	-1

After:

	A	B
0	10	7
1	0	-3
2	30	8
3	0	15
4	50	-1

```

[30]: import numpy as np

arr = np.array([10, 20, 30, 40, 50])

third_element = arr[2]

print(third_element)

```

30

```

[31]: import numpy as np

arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

```

```

# Retrieve element at row index 1 and column index 2
element = arr_2d[1, 2]

print(element)

```

6

```

[32]: import numpy as np

arr = np.array([3, 8, 2, 10, 5, 7])

# Create a boolean mask for elements greater than 5
mask = arr > 5

# Apply the mask to extract elements
result = arr[mask]

print(result)

```

[8 10 7]

```

[33]: import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])

sliced_arr = arr[2:6] # 6 is exclusive, so include 5 by using 6

print(sliced_arr)

```

[3 4 5 6]

```

[34]: import numpy as np

arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

# Slice the sub-array [[2, 3], [5, 6]]
sub_array = arr_2d[0:2, 1:3] # Rows 0 to 1 (inclusive) and Columns 1 to 2
                             ↪ (inclusive)

print(sub_array)

```

[[2 3]
[5 6]]

```
[35]: import numpy as np

def extract_elements(arr, indices):
    """
    Extract elements from a 2D NumPy array based on a given array of row-column
    ↪indices.

    Parameters:
    arr (numpy.ndarray): The input 2D array.
    indices (numpy.ndarray): A 2D array of shape (N, 2) where each row is
    ↪(row_index, col_index).

    Returns:
    numpy.ndarray: An array of extracted elements.
    """
    return arr[indices[:, 0], indices[:, 1]] # Use fancy indexing to extract
    ↪elements

# Example usage
arr_2d = np.array([[10, 20, 30],
                   [40, 50, 60],
                   [70, 80, 90]])

indices = np.array([[0, 2], # Extract element at (0,2) -> 30
                   [1, 1], # Extract element at (1,1) -> 50
                   [2, 0]]) # Extract element at (2,0) -> 70

# Extract elements
result = extract_elements(arr_2d, indices)

print(result)
```

[30 50 70]

```
[36]: import numpy as np

def filter_greater_than(arr, threshold):
    """
    Filters elements greater than a given threshold from a 1D NumPy array.

    Parameters:
    arr (numpy.ndarray): The input 1D array.
    threshold (int or float): The threshold value.

    Returns:
    numpy.ndarray: A new array containing elements greater than the threshold.
    """
```

```

    return arr[arr > threshold]  # Boolean indexing to filter elements

# Example usage
arr = np.array([3, 8, 1, 10, 5, 7])
threshold = 5

result = filter_greater_than(arr, threshold)
print(result)

```

[8 10 7]

```

[37]: def filter_greater_than(arr, threshold):
        return arr[arr > threshold]

arr = np.array([3, 8, 1, 10, 5, 7])
print(filter_greater_than(arr, 5))  # Output: [8 10 7]

```

[8 10 7]

```

[38]: def extract_elements_3d(arr, x_idx, y_idx, z_idx):
        return arr[x_idx, y_idx, z_idx]

arr_3d = np.arange(27).reshape(3, 3, 3)
x_idx = np.array([0, 1, 2])
y_idx = np.array([1, 2, 0])
z_idx = np.array([2, 0, 1])

print(extract_elements_3d(arr_3d, x_idx, y_idx, z_idx))

```

[5 15 19]

```

[39]: def filter_two_conditions(arr, condition1, condition2):
        return arr[np.logical_and(condition1, condition2)]

arr = np.array([3, 8, 1, 10, 5, 7])
result = filter_two_conditions(arr, arr > 2, arr < 9)
print(result)  # Output: [3 8 5 7]

```

[3 8 5 7]

```

[40]: def extract_elements_by_indices(arr, row_indices, col_indices):
        return arr[row_indices, col_indices]

arr_2d = np.array([[10, 20, 30],
                   [40, 50, 60],
                   [70, 80, 90]])

row_indices = np.array([0, 1, 2])

```

```
col_indices = np.array([2, 1, 0])

print(extract_elements_by_indices(arr_2d, row_indices, col_indices)) # Output:
↳ [30 50 70]
```

```
[30 50 70]
```

```
[42]: arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
      arr += 5
      print(arr)
```

```
[[ 6  7  8]
 [ 9 10 11]
 [12 13 14]]
```

```
[43]: arr1 = np.array([[2, 3, 4]]) # Shape (1,3)
      arr2 = np.array([[1, 2, 3, 4],
                       [5, 6, 7, 8],
                       [9, 10, 11, 12]]) # Shape (3,4)

      result = arr1.T * arr2 # Broadcasting
      print(result)
```

```
[[ 2  4  6  8]
 [15 18 21 24]
 [36 40 44 48]]
```

```
[44]: arr1 = np.array([[2, 3, 4, 5]]) # Shape (1,4)
      arr2 = np.array([[1, 2, 3],
                       [4, 5, 6],
                       [7, 8, 9],
                       [10, 11, 12]]) # Shape (4,3)

      result = arr1.T * arr2 # Broadcasting
      print(result)
```

```
[[ 2  4  6]
 [12 15 18]
 [28 32 36]
 [50 55 60]]
```

```
[45]: import numpy as np
      arr1 = np.array([[1],
                       [2],
                       [3]]) # Shape (3,1)

      arr2 = np.array([[10, 20, 30]]) # Shape (1,3)
```

```
# Add using broadcasting
result = arr1 + arr2

print(result)
```

```
[[11 21 31]
 [12 22 32]
 [13 23 33]]
```

```
[47]: import numpy as np

arr1 = np.array([ [2,7,8],
                  [8,9,5]])

arr2 = np.array([ [5,8],
                  [9,8]])

arr2_reshaped = arr2[:, :1]

result = arr1*arr2_reshaped

print(result)
```

```
[[10 35 40]
 [72 81 45]]
```

```
[48]: import numpy as np
arr = np.array([ [1, 2, 3],
                 [4, 5, 6]])

column_mean = np.mean(arr, axis=0)

print(column_mean)
```

```
[2.5 3.5 4.5]
```

```
[1]: import numpy as np

arr = np.array([ [1, 2, 3], [4, 5, 6]])
row_max = np.max(arr, axis=1)

print(row_max)
```

```
[3 6]
```

```
[2]: import numpy as np

arr = np.array([ [1, 2, 3], [4, 5, 6]])
```

```
col_max_indices = np.argmax(arr, axis=0)

print(col_max_indices)
```

```
[1 1 1]
```

```
[3]: import numpy as np

def moving_sum(arr, window_size=2):

    return np.convolve(arr, np.ones(window_size, dtype=int), 'valid')

arr = np.array([[1, 2, 3], [4, 5, 6]])
moving_sums = np.apply_along_axis(moving_sum, axis=1, arr=arr)

print(moving_sums)
```

```
[[ 3  5]
 [ 9 11]]
```

```
[4]: import numpy as np

arr = np.array([[2, 4, 6], [3, 5, 7]])

# Check if all elements in each column are even
col_even_check = np.all(arr % 2 == 0, axis=0)

print(col_even_check)
```

```
[False False False]
```

```
[5]: import numpy as np

original_array = np.array([1, 2, 3, 4, 5, 6])

def reshape_array(arr, m, n):
    if arr.size != m * n:
        raise ValueError("Cannot reshape array of size {} into shape ({}).".
            ↪format(arr.size, m, n))
    return arr.reshape(m, n)

# Example: Reshaping into a 2x3 matrix
reshaped_matrix = reshape_array(original_array, 2, 3)

print(reshaped_matrix)
```

```
[[1 2 3]
 [4 5 6]]
```

```
[6]: import numpy as np

def flatten_matrix(matrix):
    return matrix.flatten()

input_matrix = np.array([[1, 2, 3], [4, 5, 6]])
flattened_array = flatten_matrix(input_matrix)

print(flattened_array)
```

```
[1 2 3 4 5 6]
```

```
[7]: import numpy as np

def concatenate_arrays(arr1, arr2, axis=0):
    return np.concatenate((arr1, arr2), axis=axis)

array1 = np.array([[1, 2], [3, 4]])
array2 = np.array([[5, 6], [7, 8]])

concat_axis0 = concatenate_arrays(array1, array2, axis=0)
print("Concatenation along axis 0:\n", concat_axis0)

concat_axis1 = concatenate_arrays(array1, array2, axis=1)
print("\nConcatenation along axis 1:\n", concat_axis1)
```

Concatenation along axis 0:

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

Concatenation along axis 1:

```
[[1 2 5 6]
 [3 4 7 8]]
```

```
[2]: import numpy as np

split_rows = split_array(original_array, 3, axis=0)
print("Split along rows:", split_rows)

# Split along columns (axis=1)
split_cols = split_array(original_array, 3, axis=1)
print("Split along columns:", split_cols)

# Insert and delete example
original_array_1d = np.array([1, 2, 3, 4, 5])
```

```

indices_to_insert = [2, 4]
values_to_insert = [10, 11]
indices_to_delete = [1, 3]

modified_array = insert_and_delete(original_array_1d, indices_to_insert,
    ↪ values_to_insert, indices_to_delete)
print("Modified array:", modified_array)

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[2], line 2
      1 import numpy as np
----> 2 split_rows = split_array(original_array, 3, axis=0)
      3 print("Split along rows:", split_rows)
      5 # Split along columns (axis=1)

NameError: name 'split_array' is not defined

```

```

[3]: import numpy as np

# Create an array `arr1` with random integers
arr1 = np.random.randint(1, 100, size=10) # Random integers between 1 and 100

# Create an array `arr2` with integers from 1 to 10
arr2 = np.arange(1, 11)

# Perform element-wise addition
result = arr1 + arr2

print("arr1:", arr1)
print("arr2:", arr2)
print("Result of element-wise addition:", result)

```

```

arr1: [69 27 92  5 13 57 79 40  1 69]
arr2: [ 1  2  3  4  5  6  7  8  9 10]
Result of element-wise addition: [70 29 95  9 18 63 86 48 10 79]

```

```

[4]: import numpy as np

# Create arr1 with sequential integers from 10 to 1
arr1 = np.arange(10, 0, -1)

# Create arr2 with integers from 1 to 10
arr2 = np.arange(1, 11)

# Perform element-wise subtraction

```

```
result = arr1 - arr2

print("arr1:", arr1)
print("arr2:", arr2)
print("Result of element-wise subtraction:", result)
```

```
arr1: [10  9  8  7  6  5  4  3  2  1]
arr2: [ 1  2  3  4  5  6  7  8  9 10]
Result of element-wise subtraction: [ 9  7  5  3  1 -1 -3 -5 -7 -9]
```

```
[5]: import numpy as np

# Create an array `arr1` with random integers
arr1 = np.random.randint(1, 20, size=5) # Random integers between 1 and 20

# Create an array `arr2` with integers from 1 to 5
arr2 = np.arange(1, 6)

# Perform element-wise multiplication
result = arr1 * arr2

print("arr1:", arr1)
print("arr2:", arr2)
print("Result of element-wise multiplication:", result)
```

```
arr1: [15  2 11  6 17]
arr2: [1 2 3 4 5]
Result of element-wise multiplication: [15  4 33 24 85]
```

```
[6]: import numpy as np

# Create arr1 with even integers from 2 to 10
arr1 = np.arange(2, 11, 2) # [2, 4, 6, 8, 10]

# Create arr2 with integers from 1 to 5
arr2 = np.arange(1, 6) # [1, 2, 3, 4, 5]

# Perform element-wise division
result = arr1 / arr2

print("arr1:", arr1)
print("arr2:", arr2)
print("Result of element-wise division:", result)
```

```
arr1: [ 2  4  6  8 10]
arr2: [1 2 3 4 5]
Result of element-wise division: [2. 2. 2. 2. 2.]
```

```
[7]: import numpy as np

# Create arr1 with integers from 1 to 5
arr1 = np.arange(1, 6) # [1, 2, 3, 4, 5]

# Create arr2 with the same numbers reversed
arr2 = arr1[::-1] # [5, 4, 3, 2, 1]

# Perform element-wise exponentiation
result = arr1 ** arr2 # Equivalent to np.power(arr1, arr2)

print("arr1:", arr1)
print("arr2:", arr2)
print("Result of element-wise exponentiation:", result)
```

```
arr1: [1 2 3 4 5]
arr2: [5 4 3 2 1]
Result of element-wise exponentiation: [ 1 16 27 16  5]
```

```
[9]: import numpy as np

def count_substring(arr, substring):
    # Use np.char.count to count occurrences of the substring in each element
    return np.char.count(arr, substring).sum()

# Sample array
arr = np.array(['hellp', 'world', 'hello', 'numpy', 'hello'])

# Example: Count occurrences of 'o'
substring = 'o'
count = count_substring(arr, substring)

print(f"Occurrences of '{substring}':", count)
```

```
Occurrences of 'o': 3
```

```
[10]: import numpy as np
import re

def extract_uppercase(arr):
    """Extracts uppercase characters from each string in a NumPy array."""
    return np.array([''.join(re.findall(r'[A-Z]', s)) for s in arr])

# Example usage
arr = np.array(['Hello', 'World', 'OpenAI', 'GPT'])
result = extract_uppercase(arr)
print(result)
```

```
['H' 'W' 'OAI' 'GPT']
```

```
[11]: import numpy as np

def replace_substring(arr, old_sub, new_sub):
    """Replaces occurrences of a substring in a NumPy array of strings with a
    ↪ new string."""
    return np.char.replace(arr, old_sub, new_sub)

# Example usage
arr = np.array(['apple', 'banana', 'grape', 'pineapple'])
result = replace_substring(arr, 'apple', 'orange')
print(result)
```

```
['orange' 'banana' 'grape' 'pineorange']
```

```
[12]: import numpy as np

def longest_string_length(arr):
    """Finds the length of the longest string in a NumPy array."""
    return max(np.char.str_len(arr))

# Example usage
arr = np.array(['apple', 'banana', 'grape', 'pineapple'])
print(longest_string_length(arr)) # Output: 9
```

9

```
[13]: import numpy as np

# Generate dataset
data = np.random.randint(1, 1001, 100)

# Compute statistics
mean = np.mean(data)
median = np.median(data)
variance = np.var(data)
std_dev = np.std(data)

print(f"Mean: {mean}, Median: {median}, Variance: {variance}, Std Dev: ↪
    ↪ {std_dev}")
```

```
Mean: 579.41, Median: 588.0, Variance: 73934.68190000001, Std Dev:
271.90932661459044
```

```
[14]: data = np.random.uniform(1, 100, 50)

percentile_25 = np.percentile(data, 25)
```

```
percentile_75 = np.percentile(data, 75)

print(f"25th Percentile: {percentile_25}, 75th Percentile: {percentile_75}")
```

25th Percentile: 35.64427738331796, 75th Percentile: 85.18626486718645

```
[15]: arr1 = np.random.rand(10) # Random dataset 1
      arr2 = np.random.rand(10) # Random dataset 2

      correlation = np.corrcoef(arr1, arr2)[0, 1]
      print(f"Correlation Coefficient: {correlation}")
```

Correlation Coefficient: -0.5566905032467906

```
[16]: A = np.array([[1, 2], [3, 4]])
      B = np.array([[5, 6], [7, 8]])

      result = np.dot(A, B)
      print(result)
```

[[19 22]
 [43 50]]

```
[17]: arr = np.array([10, 20, 30, 40, 50])
      index = np.where(arr == 30)
      print(f"Index of 30: {index[0][0]}")
```

Index of 30: 2

```
[18]: arr = np.random.randint(1, 100, 10)
      sorted_arr = np.sort(arr)
      print(sorted_arr)
```

[10 17 37 50 56 67 69 70 78 89]

```
[19]: arr = np.array([12, 25, 6, 42, 8, 30])
      filtered = arr[arr > 20]
      print(filtered)
```

[25 42 30]

```
[20]: arr = np.array([1, 5, 8, 12, 15])
      filtered = arr[arr % 3 == 0]
      print(filtered)
```

[12 15]

```
[21]: arr = np.array([10, 20, 30, 40, 50])
      filtered = arr[(arr >= 20) & (arr <= 40)]
      print(filtered)
```

[20 30 40]

```
[22]: arr = np.array([1, 2, 3])
      print(arr.dtype.byteorder)
```

=

```
[23]: import numpy as np

      arr = np.array([1, 2, 3], dtype=np.int32)
      arr.byteswap(inplace=True)
      print(arr)
```

[16777216 33554432 50331648]

```
[24]: arr = np.array([1, 2, 3], dtype=np.int32)
      swapped_arr = arr.newbyteorder()
      print(swapped_arr)
```

[16777216 33554432 50331648]

```
[25]: arr = np.array([1, 2, 3], dtype=np.int32)
      if arr.dtype.byteorder == '=': # Native byte order
          swapped_arr = arr.newbyteorder()
      print(swapped_arr)
```

[16777216 33554432 50331648]

```
[26]: arr = np.array([1, 2, 3], dtype=np.int32)
      print(f"Byte order: {arr.dtype.byteorder}")
```

Byte order: =

```
[27]: arr1 = np.arange(1, 11)
      copy_arr = arr1.copy()
      copy_arr[0] = 99
      print(arr1) # Unchanged
      print(copy_arr) # Modified
```

[1 2 3 4 5 6 7 8 9 10]
[99 2 3 4 5 6 7 8 9 10]

```
[28]: matrix = np.random.randint(1, 100, (3, 3))
      view_slice = matrix[:2, :2] # Extracting a view
```

```
view_slice[0, 0] = 999
print(matrix) # Original matrix is modified
```

```
[[999  20  34]
 [ 39  87  89]
 [ 47  59   8]]
```

```
[29]: array_a = np.arange(1, 13).reshape(4, 3)
view_b = array_a[:2, :]
view_b += 5
print(array_a) # Original array is modified
```

```
[[ 6  7  8]
 [ 9 10 11]
 [ 7  8  9]
 [10 11 12]]
```

```
[30]: orig_array = np.arange(1, 9).reshape(2, 4)
reshaped_view = orig_array.reshape(4, 2)
reshaped_view[0, 0] = 999
print(orig_array) # Original array is modified
```

```
[[999  2  3  4]
 [ 5  6  7  8]]
```

```
[31]: data = np.random.randint(1, 10, (3, 4))
data_copy = data[data > 5].copy()
data_copy[0] = 99
print(data) # Unchanged
```

```
[[2 5 2 3]
 [7 2 3 6]
 [5 7 5 6]]
```

```
[32]: A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

sum_matrix = A + B
diff_matrix = A - B

print(sum_matrix)
print(diff_matrix)
```

```
[[ 6  8]
 [10 12]]
[[-4 -4]
 [-4 -4]]
```

```
[33]: C = np.random.randint(1, 10, (3, 2))
      D = np.random.randint(1, 10, (2, 4))
      product = np.dot(C, D)
      print(product)
```

```
[[33 12 48 60]
 [ 7  3  7 10]
 [13  5 17 22]]
```

```
[34]: E = np.random.randint(1, 10, (3, 4))
      transpose_E = E.T
      print(transpose_E)
```

```
[[9 6 9]
 [2 7 9]
 [2 2 7]
 [2 5 3]]
```

```
[35]: F = np.random.randint(1, 10, (3, 3))
      det_F = np.linalg.det(F)
      print(det_F)
```

```
-15.999999999999998
```

```
[36]: G = np.random.randint(1, 10, (3, 3))
      inverse_G = np.linalg.inv(G)
      print(inverse_G)
```

```
[[ 0.104  0.064 -0.128]
 [ 0.48  -0.32 -0.36 ]
 [-0.472  0.248  0.504]]
```

```
[ ]:
```

```
[ ]:
```